



Mesmo Sinal, Calibrações Diferentes: Um Índice de Tom das Atas do Copom com Múltiplos LLMs

Comparação entre Gemini Flash Lite, Claude Haiku 4.5 e GPT-4.1-mini

Vitor Wilher¹

Luiz Henrique Barbosa Filho²

25 de abril de 2026

Versão 1.0

¹Bacharel e Mestre em Economia pela UFF, Candidato ao PhD em Economia pela EPGE/FGV. É Especialista em Ciências de Dados e Inteligência Artificial Generativa pela PUC-Rio. Atualmente, exerce a função de Data Tech Lead na Análise Macro (<http://analisemacro.com.br>). Saiba mais em <https://github.com/vitorwilher>.

²Bacharel em Ciência e Economia e em Ciências Contábeis pela Universidade Federal de Alfenas (UNIFAL). Cientista de Dados focado em dados econômicos, financeiros e contábeis, com ênfase em séries temporais, previsões e inferência causal usando R e Python. Atualmente, atua como Cientista de Dados e tutor de cursos na Análise Macro (<http://analisemacro.com.br>). Saiba mais em <https://www.linkedin.com/in/luiz-henrique-barbosa-filho/>.

Resumo. A comunicação dos bancos centrais é, em si, um instrumento de política monetária, e as atas do Comitê de Política Monetária (Copom) do Banco Central do Brasil concentram, em escolhas sutis de linguagem, informação relevante sobre a direção futura da Selic. Este trabalho constrói um Índice de Tom das atas do Copom utilizando três modelos de linguagem de grande porte (LLMs) — Gemini Flash Lite (Google), Claude Haiku 4.5 (Anthropic) e GPT-4.1-mini (OpenAI) — em um *pipeline* reproduzível em Python, com saída estruturada via Pydantic e cache local incremental que torna a publicação contínua do índice viável a custo marginal próximo de zero. O mesmo *prompt* é aplicado aos mesmos textos pelos três provedores, e os *scores* brutos são calibrados em pontos percentuais equivalentes da Selic via regressão linear simples. A robustez do exercício é testada em **três camadas complementares de validação empírica**: inferência formal *in-sample* (com erros-padrão, *p*-valores e intervalos de confiança para $\hat{\beta}$), *holdout* das últimas seis reuniões e validação cruzada *walk-forward* sobre toda a amostra. Para o histórico iniciado na reunião 232 do Copom (agosto de 2020), o GPT-4.1-mini lidera tanto *in-sample* ($R^2 \approx 0,66$) quanto *out-of-sample* na *walk-forward* (RMSE 0,357, cerca de 32% melhor que o baseline léxico), seguido do Claude Haiku 4.5 ($R^2 \approx 0,43$, com a maior sensibilidade *in-sample*, $\hat{\beta} \approx +0,62$) e do Gemini Flash Lite ($R^2 \approx 0,35$, com a leitura mais conservadora); Claude e Gemini empatam *out-of-sample*, evidenciando que maior sensibilidade *in-sample* não compra poder preditivo. As correlações entre os *scores* brutos (0,67–0,78) revelam que os três modelos **concordam sobre a direção** do tom — qual ata é mais *hawkish* (sinaliza juros mais altos) ou *dovish* (sinaliza juros mais baixos) — mas **divergem sobre a intensidade** com que essa direção se traduz em variação da Selic: a sensibilidade $\hat{\beta}$ varia de +0,36 a +0,62 p.p. por unidade de *score*, uma diferença de mais de 70%. Para classificar viradas de ciclo monetário, qualquer um dos três modelos serve; para usar o índice como variável quantitativa em um modelo macroeconômico — uma regra de Taylor, por exemplo — a escolha do provedor altera o resultado de forma material.

Palavras-chave: Política Monetária; Banco Central do Brasil; Copom; Modelos de Linguagem de Grande Porte; Análise de Texto; Índice de Tom *Hawkish-Dovish*; Selic.

Códigos JEL: E52, E58, C45, C55.

Abstract. Central bank communication is, in itself, a monetary policy instrument, and the minutes of Brazil’s Monetary Policy Committee (*Copom*) concentrate, in subtle language choices, meaningful information about the future path of the Selic rate. This paper builds a Tone Index of Copom minutes using three large language models (LLMs) — Gemini Flash Lite (Google), Claude Haiku 4.5 (Anthropic), and GPT-4.1-mini (OpenAI) — within a reproducible Python *pipeline* featuring Pydantic-based structured output and incremental local caching, which makes continuous publication of the index viable at near-zero marginal cost. The same *prompt* is applied to the same texts by all three providers, and the resulting raw *scores* are calibrated in basis-points-equivalent of the Selic rate via simple linear regression. The exercise is validated through **three complementary empirical layers**: formal *in-sample* inference (with standard errors, *p*-values, and confidence intervals for $\hat{\beta}$), *holdout* of the last six meetings, and *walk-forward* cross-validation over the full sample. Over the sample starting from Copom meeting 232 (August 2020), GPT-4.1-mini leads both *in-sample* ($R^2 \approx 0.66$) and *out-of-sample* in *walk-forward* (RMSE 0.357, around 32% better than the lexicon baseline), followed by Claude Haiku 4.5 ($R^2 \approx 0.43$, with the highest *in-sample* sensitivity, $\hat{\beta} \approx +0.62$) and Gemini Flash Lite ($R^2 \approx 0.35$, with the most conservative reading); Claude and Gemini tie *out-of-sample*, showing that higher *in-sample* sensitivity does not buy predictive power. Correlations among the raw *scores* (0.67–0.78) reveal that the three models **agree on the direction** of the tone — which minutes are more *hawkish* (signaling higher interest rates) or more *dovish* (signaling lower interest rates) — but **diverge on the intensity** with which this direction translates into Selic-rate variation: the sensitivity $\hat{\beta}$ ranges from +0.36 to +0.62 percentage points per unit of *score*, a gap of more than 70%. For classifying turning points in the monetary cycle, any of the three models suffices; for using the index as a quantitative variable in a macroeconomic model — a Taylor rule, for instance — the choice of provider materially changes the outcome.

Keywords: Monetary Policy; Central Bank of Brazil; Copom; Large Language Models; Text Analysis; Hawkish-Dovish Tone Index; Selic.

JEL Codes: E52, E58, C45, C55.

Índice

1	Introdução	4
2	Revisão da Literatura	4
3	Metodologia e Dados	5
4	Implementação	7
4.1	Configuração do Ambiente	7
4.2	Coleta e Pré-processamento das Atas	8
4.3	Inferência: Score de Tom por LLM	13
4.4	Calibração e Visualização	19
5	Análise dos Resultados	24
6	Surpresa de Comunicação (Z-Score)	29
7	Próximos Passos	31
8	Conclusão	32
9	Referências	33

1 Introdução

A comunicação dos bancos centrais é uma ferramenta de política monetária por si só. As atas das reuniões do Comitê de Política Monetária (Copom) do Banco Central do Brasil (BCB) são documentos densos, onde a escolha de cada palavra — “cautela”, “parcimônia”, “vigilância” — carrega um sinal sobre a direção futura da taxa Selic. Analistas de mercado dedicam horas para decifrar essa linguagem, buscando antecipar os próximos passos do comitê.

E se pudéssemos automatizar essa leitura, transformando o texto subjetivo em um indicador quantitativo? Este exercício demonstra como utilizar **LLMs** — especificamente o **Gemini Flash Lite** (Google), o **Claude Haiku 4.5** (Anthropic) e o **GPT-4.1-mini** (OpenAI), em paralelo — para construir um **Índice de Tom das atas do Copom**. O objetivo é duplo: criar um indicador que não apenas mede o sentimento (*hawkish* vs. *dovish*) e o expressa em unidades diretamente comparáveis à taxa Selic, mas também **comparar o ajuste de três provedores** sobre o mesmo *prompt* e os mesmos textos.

2 Revisão da Literatura

A interpretação da comunicação dos bancos centrais como instrumento de política monetária é hoje consenso na macroeconomia moderna. Blinder et al. (2008) sintetizam a literatura mostrando que a comunicação reduz a incerteza sobre a função de reação do banco central, ancora expectativas e — quando bem calibrada — torna a transmissão da política mais eficaz. Em linha com a perspectiva canônica de Woodford (2003), o que o banco central comunica sobre sua função de reação futura pode ser tão importante quanto a decisão corrente sobre o instrumento. A consequência empírica é direta: se a comunicação carrega informação econômica relevante, então traduzi-la em medidas quantitativas tempestivas é, em si, um problema de pesquisa de primeira ordem.

A operacionalização desse *insight* em medidas de **tom** dos textos passou por três ondas metodológicas razoavelmente discerníveis. A primeira, baseada em **dicionários**, conta a frequência relativa de termos *hawkish* e *dovish* em relação a um vocabulário de referência. O dicionário financeiro de Loughran e McDonald (2011) é a referência canônica do campo e ainda hoje serve de *baseline* em estudos comparativos. Apel e Blix Grimaldi (2012), em trabalho seminal usando atas do Riksbank, mostram que o conteúdo informacional das minutas se concentra em seções específicas de diagnóstico macroeconômico — observação metodológica que motiva, no presente trabalho, a extração focada das seções A (atualização da conjuntura) e B (cenários e riscos) das atas do Copom.

A segunda onda, baseada em **modelos de tópicos e aprendizado supervisionado**, busca capturar dimensões latentes do discurso que não se reduzem a contagens de palavras. Hansen et al. (2018), em contribuição publicada no *Quarterly Journal of Economics*, aplicam *Latent Dirichlet Allocation* (LDA) às transcrições do FOMC e identificam, explorando mudanças de regime de transparência, como a deliberação interna se traduz em comunicação pública. Picault e Renault (2017) propõem para o BCE um índice de tom baseado em vocabulário customizado e demonstram poder explicativo sobre decisões e expectativas. Bholat et al. (2015) documentam, em monografia do *Bank of England*, o estado das ferramentas de *text mining* aplicadas a bancos centrais. Hubert e Labondance (2021) fecham a onda mostrando que o tom da comunicação do FOMC produz efeitos de sinalização **independentes** da decisão de taxa — evidência de que tom é uma dimensão informacional separável do instrumento de política, e portanto merece ser medida em uma métrica própria.

A terceira onda, em curso, é a dos **modelos de linguagem de grande porte (LLMs)**. A virada metodológica é qualitativa: enquanto dicionários medem frequência e modelos de tópicos medem co-ocorrência, LLMs interpretam **contexto, intenção e nuance** — capacidades historicamente reservadas a leitores humanos especializados. Hansen e Kazinnik (2023), comparando o ChatGPT a leitores profissionais, mostram que LLMs de propósito geral conseguem decifrar a comunicação do *Federal Reserve* com qualidade comparável à de analistas treinados e a custo marginal próximo de zero. Em chave complementar, Hansen e McMahon (2016) fornecem a justificativa formal para medir tom em unidades comparáveis a instrumentos de política, ao tratar choques de comunicação como variável macroeconômica com efeitos identificáveis sobre produto e inflação.

Para o caso brasileiro, a literatura empírica é mais escassa, refletindo o volume comparativamente menor de pesquisa em PLN aplicado a textos em português e a dificuldades específicas do português jurídico-econômico do BCB. Caruso (2026), em relatório do Santander Brasil que adotamos como referência empírica direta, documenta que medidas de tom das atas têm poder antecipatório sobre a Selic e propõe um *pipeline* de monitoramento contínuo. O presente trabalho dialoga com essa contribuição, mas avança em duas frentes complementares. Primeiro, **substitui o instrumento de leitura**: saindo de heurísticas baseadas em dicionários e contagens para LLMs com saída estruturada via Pydantic, calibrados a uma escala de tom monetário desenhada especificamente para a comunicação do Copom. Segundo, **compara três provedores em paralelo** — Gemini, Claude e OpenAI — sobre o mesmo *prompt* e os mesmos textos, isolando efeitos atribuíveis ao modelo de efeitos atribuíveis ao *prompt* ou à amostra de texto.

A contribuição metodológica do presente trabalho se posiciona, portanto, na fronteira entre a terceira onda da literatura internacional e o monitoramento aplicado de política monetária brasileira. Oferecemos um *pipeline* reproduzível, calibrado em unidades diretamente comparáveis à Selic, e — pelo que conhecemos da literatura — pioneiro na **comparação multi-provedor** de LLMs sobre as atas do Copom.

3 Metodologia e Dados

O processo foi estruturado em quatro etapas principais, demonstrando a capacidade do Python em orquestrar desde a coleta de dados até a modelagem estatística e visualização.

1. Coleta e Pré-processamento de Dados O primeiro passo foi construir um coletor de dados automatizado. Utilizando a biblioteca `requests`, o código acessa diretamente as APIs públicas do BCB para baixar o conteúdo de todas as atas do Copom desde a reunião 232 (agosto de 2020) e a série histórica da meta Selic. Para otimizar a análise pelo LLM, aplicamos um pré-processamento inteligente: * **Limpeza de HTML**: Removemos todas as tags HTML, notas de rodapé e espaços excessivos, transformando o documento em texto puro. * **Extração de Seções Relevantes**: O sinal informacional de uma ata concentra-se nas seções de diagnóstico da conjuntura (Seção A) e balanço de riscos (Seção B). As seções seguintes, que anunciam a decisão já conhecida, são menos informativas. O código isola apenas as seções A e B, reduzindo em cerca de 50% o número de *tokens* enviados ao modelo sem perda de informação relevante.

Justificativa do recorte amostral. A escolha de iniciar a amostra na reunião 232 (agosto de 2020) é metodológica, não acidental, e responde a três critérios de comparabilidade. Primeiro, **consistência da estrutura textual**: a numeração padronizada em seções A (atualização da conjuntura),

B (cenários e riscos) e C (decisão) das atas foi consolidada a partir de 2017 e estabilizou-se definitivamente em 2020 — atas anteriores adotam estruturas heterogêneas que invalidariam a regra de extração focada em A+B descrita acima e contaminariam o *prompt* enviado aos LLMs com texto da decisão (justamente o que tentamos isolar). Segundo, **regime monetário único**: a amostra cobre uma única configuração de meta de inflação (com a meta contínua centrada em 3,0% a partir de 2024 e convergência gradual desde 2018) e duas gestões executivas do BCB com regra de Taylor implícita comparável (Campos Neto / Galípolo) — evitando quebras estruturais na função de reação que poderiam corromper a interpretação de $\hat{\beta}$. Terceiro, **ciclo monetário completo**: a janela cobre afrouxamento pandêmico em 2020, aperto agressivo em 2021–2022, ciclo de cortes em 2023, e novo ciclo de aperto a partir de 2024 — variação suficiente em ΔSelic para identificar $\hat{\beta}$ com precisão, como evidenciam os t -estatísticos $\geq 3,6$ para os quatro modelos calibrados (Tabela~1). Estender o exercício a regimes anteriores (pré-2017 ou pré-tripé macroeconômico) é uma direção natural de pesquisa, mas exige modelagem explícita de quebra estrutural — registrada nos *Próximos Passos*.

2. Geração do Score Bruto via LLM (Engenharia de Prompt) Esta é a etapa central. Os mesmos textos passam por **três provedores em paralelo** — `gemini-flash-lite-latest` (Google), `claude-haiku-4-5` (Anthropic) e `gpt-4.1-mini` (OpenAI) — através do LangChain, com **exatamente o mesmo prompt**. O modelo recebe a persona de um “economista sênior do BCB” e uma escala de pontuação detalhada, que vai de -3.0 (fortemente *dovish*) a +3.0 (fortemente *hawkish*). O *prompt* inclui âncoras explícitas para cada faixa de score (ex: “-2.0: balanço de riscos assimétrico para baixo; desinflação clara”) e a instrução crucial de ignorar a decisão da taxa já anunciada, focando apenas no tom implícito do texto. No caso do Claude, o bloco de instruções é marcado com `cache_control: ephemeral` — *prompt caching* nativo da Anthropic — de modo que, se atendido o tamanho mínimo, a partir da segunda chamada o cabeçalho é lido do cache, reduzindo custo de input e latência.

Para garantir a robustez da saída, utilizamos a funcionalidade de **saída estruturada (Pydantic)**. Cada LLM é forçado a retornar sua resposta em um formato JSON pré-definido, contendo apenas um número *float*. Isso elimina a necessidade de *parsing* frágil de texto e garante que o resultado seja diretamente utilizável — e diretamente comparável entre provedores.

3. Calibração OLS e Validação Empírica em Três Camadas Os scores brutos (de -3 a +3) são qualitativos. Para dar-lhes um significado econômico, calibramos **uma regressão linear simples (OLS) por modelo** — os três LLMs e um **baseline léxico** de contagem de palavras *hawkish/dovish* em português (no espírito de Loughran e McDonald (2011) e Apel e Blix Grimaldi (2012), incluído como piso de comparação metodológico). A variável dependente é a variação da Selic em cada reunião (em pontos percentuais); a explicativa é o *score* bruto. Cada um dos quatro modelos ganha seus próprios $\hat{\alpha}$, $\hat{\beta}$ e R^2 , tornando direta a comparação de poder explicativo.

$$\Delta\text{Selic}_t = \alpha + \beta \cdot \text{Score}_t + \varepsilon_t$$

A robustez do exercício é então avaliada em **três camadas complementares**: (i) **inferência *in-sample*** via `statsmodels.OLS`, reportando erros-padrão, t -estatísticas, p -valores e intervalos de confiança de 95% para $\hat{\beta}$ — o que permite afirmar quando uma diferença entre modelos é estatisticamente robusta e quando não é; (ii) ***holdout fora-da-amostra*** das últimas seis reuniões, com $\hat{\alpha}$ e $\hat{\beta}$ calibrados apenas no conjunto de treino e RMSE/MAE medidos sobre o holdout; (iii) **validação cruzada *walk-forward*** com janela expansiva (treino mínimo de vinte atas), em que cada reunião subsequente é prevista pelo modelo OLS treinado em todas as anteriores, varrendo o ciclo monetário inteiro. Os três exercícios cumprem papéis complementares: o primeiro estabelece

a inferência estatística; o segundo expõe o desempenho preditivo numa janela específica (a mais recente); o terceiro testa a generalização ao longo de toda a amostra.

4. Cache Incremental e Reprodutibilidade Como o paper é pensado para ser republicado a cada nova ata do Copom, o *pipeline* foi desenhado para ser **incremental** — só recomputa o que efetivamente mudou. Três caches locais sustentam essa propriedade, todos no mesmo diretório do `.qmd`:

- **atas_cache.json** — texto pré-processado de cada ata (seções A+B), indexado por número de reunião. Como atas do Copom não são revisadas após publicação, o cache é **permanente**: do segundo *render* em diante, o BCB é consultado apenas para descobrir se há ata nova e baixá-la, caso exista.
- **selic_cache.json** — espelho da série SGS 432 do BCB. A API é sempre tentada primeiro (com `Retry` e *backoff* exponencial); o cache entra em cena apenas como *fallback* em caso de falha de rede, garantindo que o *render* não quebre por instabilidade do servidor.
- **scores_{gemini,claude,openai}_cache.csv** — um arquivo por provedor com pares (`nro_reuniao`, `score`). O *loop* de inferência consulta o CSV antes de chamar cada API; apenas reuniões inéditas pagam custo de LLM. Para reprocessar do zero (mudança de *prompt*, troca de modelo, ajuste na extração das seções), basta apagar o CSV correspondente.

Esse desenho **complementa** o *prompt caching* da Anthropic descrito na etapa 2 — não o substitui. O cache local atua **antes** da chamada (evita a requisição quando o score já foi computado); o *prompt caching* atua **dentro** das chamadas que efetivamente acontecem (barateia o *input* repetido). Em conjunto, viabilizam a publicação contínua do índice: a cada nova ata, o pipeline incorre em **uma chamada por provedor** e todo o histórico anterior é reaproveitado.

A implementação a seguir percorre as quatro etapas descritas na ordem em que foram apresentadas — coleta de atas e Selic, *scoring* via LLM (mais o baseline léxico), calibração OLS com validação em três camadas, e *cache* incremental atravessando as três etapas anteriores —, com cada bloco de código antecedido por uma breve nota sobre o que ele faz e por que está estruturado dessa forma.

4 Implementação

4.1 Configuração do Ambiente

O primeiro passo é instalar as dependências do *pipeline*. O conjunto reúne os adaptadores `LangChain` dos três provedores (Google, Anthropic e OpenAI), o *parser* HTML `beautifulsoup4` para limpar o conteúdo das atas, `statsmodels` para a regressão OLS com inferência (erros-padrão, *p*-valores e intervalos de confiança), `scikit-learn` como suporte adicional e `plotnine` para os gráficos no estilo *ggplot*.

```
# Instalação
%pip install langchain-google-genai langchain-anthropic langchain-openai langchain-core beautifulsoup4 scikit-learn
```

Com as dependências instaladas, importamos as bibliotecas que sustentam o *pipeline*. `requests` e `BeautifulSoup` cuidam da coleta de dados; `pandas` e `numpy` fazem a manipulação tabular; `statsmodels` conduz a regressão OLS com inferência completa; `pydantic` define o *schema* de saída estruturada que cada LLM deve respeitar; `plotnine` produz os gráficos. Um pequeno *fallback*

na importação de `ServerError` protege contra ambientes em que o pacote `google.genai` não está exposto no nível esperado, mantendo a lógica de *retry* do Gemini funcional em qualquer instalação.

```
# Bibliotecas
import os
import re
import time
import requests
from dotenv import load_dotenv

import numpy as np
import pandas as pd
from plotnine import *
from bs4 import BeautifulSoup
from pydantic import BaseModel, Field
from sklearn.linear_model import LinearRegression
import statsmodels.api as sm

from langchain_core.documents import Document
from langchain_core.prompts import PromptTemplate
from langchain_core.messages import SystemMessage, HumanMessage
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_anthropic import ChatAnthropic
from langchain_openai import ChatOpenAI
# google.genai may not be installed as a top-level package in some envs;
# provide a safe fallback for ServerError to keep retry logic working.
try:
    from google.genai.errors import ServerError
except Exception:
    # Try common Google client exception types
    try:
        from google.api_core.exceptions import GoogleAPICallError as ServerError
    except Exception:
        try:
            # alternative package layout: google_genai
            from google_genai import errors as _gerr
            ServerError = getattr(_gerr, 'ServerError', Exception)
        except Exception:
            ServerError = Exception # final fallback
```

4.2 Coleta e Pré-processamento das Atas

As três APIs de LLM exigem chaves separadas. Para evitar credenciais em código, usamos `python-dotenv` para carregá-las de um arquivo `.env` compartilhado entre os projetos do portfólio, falhando explicitamente caso qualquer uma esteja ausente — assim o *render* é interrompido com uma mensagem clara em vez de propagar erros opacos no meio da inferência.

```
# Autenticação nas APIs (Google Gemini, Anthropic Claude, OpenAI)
load_dotenv("/Users/vitorwilher/Dropbox/Portfolio/Github_Portfolio/.env", override=True)

if not os.environ.get("GOOGLE_API_KEY"):
    raise EnvironmentError("GOOGLE_API_KEY não encontrada.")
if not os.environ.get("ANTHROPIC_API_KEY"):
```

```

    raise EnvironmentError("ANTHROPIC_API_KEY não encontrada.")
if not os.environ.get("OPENAI_API_KEY"):
    raise EnvironmentError("OPENAI_API_KEY não encontrada.")

print("GOOGLE_API_KEY, ANTHROPIC_API_KEY e OPENAI_API_KEY carregadas.")

```

Com as credenciais no lugar, definimos as **funções auxiliares de coleta**. São três endpoints públicos do BCB: **atas** (lista metadados das reuniões e nos diz qual é a última disponível), **atas_detalhes** (devolve o HTML completo de uma ata específica) e **bcddata.sgs.432** (a série diária da meta Selic). A função **_html_para_texto** limpa as tags HTML e notas de rodapé; **_extrair_secoes_ab** é o coração do pré-processamento — identifica via *regex* o início da Seção A e o início da Seção C, devolvendo apenas o intervalo entre elas (cerca de metade do conteúdo original, sem perda de informação relevante para o tom). **coletar_selic_meta** adiciona uma **Session** com **Retry/backoff** exponencial e persiste o resultado em **selic_cache.json**, que serve de *fallback* em caso de falha de rede. **selic_na_reuniao** faz a ponte entre as duas séries: dado o dia da reunião, retorna a variação da Selic em uma janela de ± 10 dias (ou zero, se a meta foi mantida).

```

# Funções auxiliares de coleta e limpeza ----
# Endpoints públicos do BCB usados neste notebook
BCB_LISTA = "https://www.bcb.gov.br/api/servico/sitebcb/copom/atas"
BCB_DETALHE = "https://www.bcb.gov.br/api/servico/sitebcb/copom/atas_detalhes"
BCB_SELIC = "https://api.bcb.gov.br/dados/serie/bcddata.sgs.432/dados"
HEADERS = {"Accept": "application/json", "User-Agent": "CopomToneIndex/2.0"}

# Reunião 232 (primeira ata que queremos coletar por padrão)
PRIMEIRA_REUNIAO_HTML = 232

def _html_para_texto(html: str) -> str:
    """Remove tags HTML, notas de rodapé e normaliza espaços."""
    soup = BeautifulSoup(html, "html.parser")
    for tag in soup.find_all(["sup", "script", "style"]):
        tag.decompose()
    return re.sub(r"\s+", " ", soup.get_text(" ", strip=True))

def _extrair_secoes_ab(texto: str, max_chars: int = 4_500) -> str:
    """
    Extraí as seções A (atualização da conjuntura) e B (cenários e riscos).
    Essas seções concentram o sinal informacional do LLM:
    • Seção A: diagnóstico macroeconômico - inflação, atividade, cenário externo
    • Seção B: balanço de riscos e projeções
    A seção C em diante anuncia a decisão já conhecida - menos informativa para o índice.
    Reduz tokens enviados ao LLM em ~50% sem perda de informação relevante.
    """
    m_a = re.search(r"A\\s*Atualiza", texto, re.IGNORECASE)
    m_c = re.search(r"C\\s*(Discuss|Decis|Voto|Condu)", texto, re.IGNORECASE)
    if m_a:
        ini = m_a.start()
        fim = m_c.start() if m_c else ini + max_chars
        return texto[ini:fim][:max_chars]
    return texto[:max_chars]

def coletar_ata(nro_reuniao: int) -> Document | None:
    """

```

Coleta uma ata via API do BCB (endpoint atas_detalhes).
 Usa requests + BeautifulSoup diretamente - sem WebBaseLoader.
 Retorna Document com texto limpo das seções A+B e metadados.
 """

```
for tentativa in range(3):
    try:
        resp = requests.get(
            BCB_DETALHE,
            params={"nro_reuniao": nro_reuniao},
            headers=HEADERS,
            timeout=30,
        )
        resp.raise_for_status()
        item = resp.json()["conteudo"][0]
        texto = _html_para_texto(item.get("textoAta", ""))
        return Document(
            page_content=_extrair_secoes_ab(texto),
            metadata={
                "nro_reuniao": item["nroReuniao"],
                "data": item["dataReferencia"][:10],
            },
        )
    except Exception as e:
        if tentativa < 2:
            time.sleep(2 ** tentativa)
        else:
            print(f" [ERRO] Ata {nro_reuniao}: {e}")
            return None
```

```
def coletar_selic_meta(data_inicio: str = "01/07/2016", timeout: int = 30, max_retries: int = 3, cache_path: str = "cache_selic.json"):
```

Coleta a série da meta Selic diária (SGS 432) e calcula a variação entre observações.
 Usa uma Session do requests com Retry/exponential backoff e salva um cache local em JSON.

Parâmetros:

- data_inicio: string no formato DD/MM/AAAA para filtrar início da série.
- timeout: tempo limite por requisição em segundos.
- max_retries: número máximo de tentativas em erros transitórios.
- cache_path: caminho para arquivo JSON de cache (será criado/atualizado após sucesso).

Retorna:

- DataFrame com colunas ['data', 'valor', 'variacao_pp'].
 """

```
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry
import json, os

session = requests.Session()
retries = Retry(
    total=max_retries,
    backoff_factor=1, # 1s, 2s, 4s ...
    status_forcelist=(429, 500, 502, 503, 504),
    allowed_methods=("GET",),
)
session.mount("https://", HTTPAdapter(max_retries=retries))

data = None
```

```

try:
    resp = session.get(
        BCB_SELIC,
        params={"formato": "json", "dataInicial": data_inicio},
        headers=HEADERS,
        timeout=timeout,
    )
    resp.raise_for_status()
    data = resp.json()
    # tenta salvar cache local (não falha a execução se erro no salvamento)
    try:
        with open(cache_path, 'w', encoding='utf-8') as f:
            json.dump(data, f, ensure_ascii=False, indent=2)
    except Exception as e:
        print('Aviso: não foi possível salvar cache da Selic:', e)
except requests.exceptions.RequestException as e:
    # Em erro de rede/timeouts, tenta carregar cache local se existir
    print('Erro ao obter Selic via API:', type(e).__name__, e)
    if os.path.exists(cache_path):
        print('Carregando Selic do cache local:', cache_path)
        with open(cache_path, 'r', encoding='utf-8') as f:
            data = json.load(f)
    else:
        # sem cache, propaga o erro para o usuário tratar
        raise

df = pd.DataFrame(data)
df["data"] = pd.to_datetime(df["data"], format="%d/%m/%Y")
df["valor"] = df["valor"].astype(float)
df = df.sort_values("data").drop_duplicates("data").reset_index(drop=True)
df["variacao_pp"] = df["valor"].diff().round(4)
return df

def selic_na_reuniao(df_selic: pd.DataFrame, data_ata: str) -> float:
    """
    Retorna a variação da Selic associada à reunião de uma ata.
    Janela de ±10 dias ao redor da data de referência da ata.
    Retorna 0.0 se a Selic foi mantida na reunião.
    """
    data = pd.to_datetime(data_ata)
    janela = df_selic[
        (df_selic["data"] >= data - pd.Timedelta(days=10)) &
        (df_selic["data"] <= data + pd.Timedelta(days=10))
    ]
    mudancas = janela[janela["variacao_pp"].abs() > 0.01]
    return float(mudancas.iloc[0]["variacao_pp"]) if len(mudancas) > 0 else 0.0

```

Com as funções no lugar, executamos a coleta. O BCB é consultado uma vez para descobrir o número da última reunião publicada; em seguida, o *loop* itera da reunião 232 até a última, aproveitando `atas_cache.json` para reuniões já vistas e baixando apenas o que faltar. O `time.sleep(0.4)` entre chamadas só é acionado em requisições reais — atas em cache são lidas instantaneamente.

```

# Coleta de dados ----
# Cache incremental das atas em JSON: as atas do Copom não mudam após publicação,
# então armazenamos o texto já pré-processado (seções A+B) por nro_reuniao e

```

```

# só consultamos o BCB para reuniões inéditas. Para reprocessar do zero
# (ex.: mudança em _extrair_secoes_ab), apague atas_cache.json - e idealmente
# também os scores*_cache.csv, que foram gerados sobre o texto antigo.
import json

CACHE_ATAS = "atas_cache.json"

# Número da última reunião disponível
ultima_ata = requests.get(
    BCB_LISTA, params={"quantidade": 1}, headers=HEADERS, timeout=20
).json()["conteudo"][0]["nroReuniao"]

if os.path.exists(CACHE_ATAS):
    with open(CACHE_ATAS, "r", encoding="utf-8") as f:
        cache_atas = json.load(f)
        print(f"Cache atas: {len(cache_atas)} ata(s) existentes.")
else:
    cache_atas = {}
    print("Sem cache de atas - todas serão baixadas do BCB.")

print(f"Última reunião: {ultima_ata}")
print(f"Coletando atas {PRIMEIRA_REUNIAO_HTML}-{ultima_ata}...\n")

documentos = []
novas_atas = 0
for num in range(PRIMEIRA_REUNIAO_HTML, ultima_ata + 1):
    chave = str(num)
    if chave in cache_atas:
        item = cache_atas[chave]
        documentos.append(Document(
            page_content=item["texto"],
            metadata={"nro_reuniao": num, "data": item["data"]},
        ))
        continue

    doc = coletar_ata(num)
    if doc:
        documentos.append(doc)
        cache_atas[chave] = {
            "data": doc.metadata["data"],
            "texto": doc.page_content,
        }
        novas_atas += 1
        print(f"    {num} | {doc.metadata['data']} | {len(doc.page_content):,} chars")
    time.sleep(0.4) # pausa só após chamadas reais ao BCB

if novas_atas > 0:
    with open(CACHE_ATAS, "w", encoding="utf-8") as f:
        json.dump(cache_atas, f, ensure_ascii=False, indent=2)
    print(f"\nCache atas atualizado: +{novas_atas} ata(s), total {len(cache_atas)}.")
elif documentos:
    print(f"\nCache atas sem mudanças ({len(cache_atas)} ata(s)).")

print(f"\nTotal: {len(documentos)} atas")

```

Em paralelo, coletamos a série diária da meta Selic. A API do BCB é a fonte primária; o cache local em JSON é tocado apenas se a rede falhar, garantindo que o *render* sobreviva a indisponibilidades momentâneas do servidor.

```
# Coleta da Selic
df_selic = coletar_selic_meta()
print(f"Selic: {len(df_selic)} observações diárias de {df_selic['data'].min().date()} a {df_selic['data'].max().date()}")
```

4.3 Inferência: Score de Tom por LLM

Com texto e Selic em mãos, partimos para o coração do exercício: a inferência LLM. Três decisões estruturais aparecem no primeiro bloco e se repetem nos outros dois provedores. Primeiro, o *schema* de saída é um modelo Pydantic (TomAta) com um único campo `score: float` — anexado ao LLM via `with_structured_output`, ele elimina o *parsing* frágil de texto livre e garante que os três modelos retornem números diretamente comparáveis. Segundo, o *prompt* é declarado uma única vez, na constante `INSTRUcoes_SISTEMA`, para garantir que Gemini, Claude e OpenAI vejam **exatamente** as mesmas instruções — qualquer divergência de ajuste será atribuível ao modelo, não ao prompt. Terceiro, o *loop* lê `scores_<provedor>_cache.csv` antes de chamar a API e registra apenas as atas inéditas — o cache é o que torna o paper sustentável a cada novo ciclo do Copom.

Começamos pelo **Gemini Flash Lite**. O `temperature=0` força o modelo a escolher sempre o *token* mais provável, maximizando o determinismo das respostas; `with_structured_output(..., method="json_schema")` usa o suporte nativo do Gemini a saídas tipadas; o `with_retry` cuida de erros transitórios (5xx do servidor) com *backoff* exponencial e *jitter*.

```
# Modelagem LLM - Gemini (Google)
# Schema de saída estruturada
# O LLM retorna um float direcional
# method='json_schema' usa a API nativa do Gemini para forçar o formato.

# Schema de saída estruturada - garante retorno de float, sem parsing frágil
class TomAta(BaseModel):
    score: float = Field(
        description=(
            "Score de tom de política monetária de -3.0 a +3.0. "
            "Negativo = dovish. Positivo = hawkish. Zero = neutro."
        )
    )

# gemini-flash-lite-latest + temperature=0 → máximo determinismo
llm = ChatGoogleGenerativeAI(model="gemini-flash-lite-latest",
                              temperature=0,
                              max_retries=2,
                              )

llm_struct = llm.with_structured_output(TomAta, method="json_schema")

# Prompt especializado em política monetária brasileira.
# Âncoras explícitas por faixa e instrução de ignorar a decisão já anunciada
# tornam o score mais consistente entre atas e minimizam variância entre chamadas.
# As instruções do sistema são extraídas como constante para reuso entre Gemini e Claude.
INSTRUcoes_SISTEMA = """\
Você é um economista especializado em política monetária do Banco Central do Brasil.
Analisar o tom de política monetária implícito no trecho de ata do Copom abaixo.

Retorne um score numérico de -3.0 a +3.0 seguindo rigorosamente esta escala:
```

- 3.0 Fortemente dovish: cortes grandes e imediatos são certeza; linguagem explicitamente acomodatória; inflação muito benigna; expectativas ancoradas; grave deterioração da atividade.
- 2.0 Dovish: balanço de riscos assimétrico para baixo; desinflação clara; espaço explícito para cortes.
- 1.0 Levemente dovish: tom inclinado ao afrouxamento, mas com cautela; convergência da inflação em curso.
- 0.0 Neutro / data-dependent: balanço simétrico; sem sinalização clara; linguagem condicional.
- +1.0 Levemente hawkish: atenção crescente à inflação; expectativas pressionadas; política deve permanecer contracionista.
- +2.0 Hawkish: inflação acima da meta com riscos altistas; expectativas desancoradas; altas sinalizadas.
- +3.0 Fortemente hawkish: inflação fora de controle; altas urgentes e certas; linguagem altamente restritiva.

Instruções:

- Avalie APENAS o tom implícito nas seções de diagnóstico e balanço de riscos.
- NÃO se baseie na decisão de taxa já anunciada na ata.
- Foque em: linguagem sobre inflação, ancoragem de expectativas, balanço de riscos e sinalização prospectiva (forward guidance).
- Use valores intermediários (ex: -1.5, +0.5) quando o tom for ambíguo.
- Seja consistente: atas com linguagem similar devem receber scores próximos.\n""

```
template = INSTRUcoes_SISTEMA + "\n\nTRECHO DA ATA:\n{DOCUMENTO}\n"
```

```
prompt = PromptTemplate.from_template(template)
```

```
chain = (
    prompt
    | llm_struct
).with_retry(
    retry_if_exception_type=(ServerError, Exception),
    wait_exponential_jitter=True,
    exponential_jitter_params={"initial": 5, "max": 120},
    stop_after_attempt=6,
)
```

```
# Inferência Gemini com cache incremental em CSV
# Carrega o cache (se existir) e chama a API APENAS para as atas faltantes.
# Quando o BCB publica uma nova ata, o próximo render infere só a nova (1 chamada).
# Para reprocessar do zero: apague scores_gemini_cache.csv.
CACHE_GEMINI = "scores_gemini_cache.csv"
```

```
if os.path.exists(CACHE_GEMINI):
    _df = pd.read_csv(CACHE_GEMINI)
    cache_gemini = dict(zip(_df["nro_reuniao"], _df["score"]))
    print(f"Cache Gemini: {len(cache_gemini)} scores existentes.")
else:
    cache_gemini = {}
    print("Sem cache Gemini - todas as atas serão inferidas.")
```

```
scores_gemini = []
datas          = []
nros           = []
```

```

erros_gemini = []
novos_gemini = 0

for i, doc in enumerate(documentos, 1):
    nro = doc.metadata["nro_reuniao"]
    data = doc.metadata["data"]
    info = f"{i}/{len(documentos)} | reunião {nro} | {data}"

    if nro in cache_gemini:
        scores_gemini.append(cache_gemini[nro])
        datas.append(data)
        nros.append(nro)
        continue

    try:
        resultado = chain.invoke({"DOCUMENTO": doc.page_content})
        score = float(np.clip(resultado.score, -3.0, 3.0))
        scores_gemini.append(score)
        datas.append(data)
        nros.append(nro)
        novos_gemini += 1
        print(f"    {info} | score: {score:+.2f}")
    except Exception as e:
        erros_gemini.append({"nro_reuniao": nro, "data": data, "erro": str(e)})
        print(f"    {info} | falhou após todos os retries: {type(e).__name__}")
        time.sleep(1.0) # pausa base entre atas (independente dos retries)

if erros_gemini:
    print(f"\n {len(erros_gemini)} ata(s) com falha: {[e['nro_reuniao'] for e in erros_gemini]}")

if novos_gemini > 0 and scores_gemini:
    pd.DataFrame({
        "nro_reuniao": nros,
        "data": datas,
        "score": scores_gemini,
    }).to_csv(CACHE_GEMINI, index=False)
    print(f"Cache Gemini atualizado: +{novos_gemini} ata(s), total {len(scores_gemini)}.")
elif scores_gemini:
    print(f"Cache Gemini sem mudanças ({len(scores_gemini)} scores).")
else:
    print("Nenhum score Gemini válido - cache NÃO salvo para permitir reproprocessamento.")

```

O **Claude Haiku 4.5** segue o mesmo molde, com um detalhe adicional: o bloco de instruções é embalado como um `SystemMessage` cujo conteúdo carrega `cache_control: {type: ephemeral}`. Isso ativa o *prompt caching* nativo da Anthropic — se o tamanho do *system prompt* atender ao mínimo exigido pelo modelo, da segunda chamada em diante a Anthropic lê o cabeçalho do cache do servidor, reduzindo custo de *input* e latência. É a segunda camada de cache descrita na metodologia, que opera **por dentro** das chamadas que de fato acontecem (enquanto o cache local em CSV evita a chamada por completo).

```

# Modelagem LLM - Claude (Anthropic)
# Mesmo schema Pydantic e mesma escala do bloco anterior; muda apenas o provider.
# claude-haiku-4-5 + temperature=0 → equivalente direto do Flash Lite no espectro Claude.
# Prompt caching (cache_control: ephemeral) marca o bloco de instruções como cacheável:
# se o tamanho do system prompt atender ao mínimo do modelo, da segunda chamada em
# diante a Anthropic lê do cache, reduzindo latência e custo de input significativamente.

```

```

llm_claude = ChatAnthropic(
    model="claude-haiku-4-5",
    temperature=0,
    max_tokens=300,
    max_retries=6,
    timeout=60.0, # evita travar indefinidamente em chamada lenta
)

llm_claude_struct = llm_claude.with_structured_output(TomAta)

def chamar_claude(documento: str) -> TomAta:
    """Invoca Claude com system+user separados; system marcado como cacheável."""
    system_blocks = [{
        "type": "text",
        "text": INSTRUcoes_SISTEMA,
        "cache_control": {"type": "ephemeral"},
    }]
    user_msg = f"TRECHO DA ATA:\n{documento}"
    return llm_claude_struct.invoke([
        SystemMessage(content=system_blocks),
        HumanMessage(content=user_msg),
    ])

# Inferência Claude com cache incremental em CSV (alinhado por nro_reuniao com Gemini).
# Carrega o cache (se existir) e chama a API APENAS para as atas faltantes.
# Para reprocessar do zero: apague scores_claude_cache.csv.
CACHE_CLAUDE = "scores_claude_cache.csv"
nros_ok = set(nros)
docs_ok = [d for d in documentos if d.metadata["nro_reuniao"] in nros_ok]

if os.path.exists(CACHE_CLAUDE):
    _df = pd.read_csv(CACHE_CLAUDE)
    cache_claude = dict(zip(_df["nro_reuniao"], _df["score"]))
    print(f"Cache Claude: {len(cache_claude)} scores existentes.")
else:
    cache_claude = {}
    print("Sem cache Claude - todas as atas serão inferidas.")

scores_claude = []
erros_claude = []
novos_claude = 0

for i, doc in enumerate(docs_ok, 1):
    nro = doc.metadata["nro_reuniao"]
    data = doc.metadata["data"]
    info = f"{i}/{len(docs_ok)} | reunião {nro} | {data}"

    if nro in cache_claude:
        scores_claude.append(cache_claude[nro])
        continue

    try:
        resultado = chamar_claude(doc.page_content)
        score = float(np.clip(resultado.score, -3.0, 3.0))
        scores_claude.append(score)

```

```

        novos_claude += 1
        print(f"      {info} | score: {score:+.2f}")
    except Exception as e:
        scores_claude.append(np.nan)
        erros_claude.append({"nro_reuniao": nro, "data": data, "erro": str(e)})
        print(f"      {info} | falhou após todos os retries: {type(e).__name__}")
    time.sleep(0.4)

if erros_claude:
    print(f"\n {len(erros_claude)} ata(s) com falha: {[e['nro_reuniao'] for e in erros_claude]}")

if novos_claude > 0 and any(pd.notna(s) for s in scores_claude):
    pd.DataFrame({
        "nro_reuniao": [d.metadata["nro_reuniao"] for d in docs_ok],
        "score":        scores_claude,
    }).to_csv(CACHE_CLAUDE, index=False)
    print(f"Cache Claude atualizado: +{novos_claude} ata(s), total {len(scores_claude)}.")
elif scores_claude:
    print(f"Cache Claude sem mudanças ({len(scores_claude)} scores).")
else:
    print("Nenhum score Claude válido - cache NÃO salvo para permitir reproprocessamento.")

```

O GPT-4.1-mini fecha o trio. A OpenAI usa *tool calling* nativo para a saída estruturada — o `with_structured_output` cuida disso transparentemente — e não expõe um mecanismo de *prompt caching* equivalente ao do Claude do lado do cliente. O cache local em CSV continua valendo, então o custo incremental por *render* é o mesmo dos outros provedores: zero quando não há ata nova. O `timeout=45.0` foi calibrado após observar episódios em que a chamada ficava pendurada silenciosamente; com o limite, o *loop* falha rápido e segue para a próxima ata.

```

# Modelagem LLM - OpenAI
# Mesmo schema Pydantic e mesma escala dos blocos anteriores.
# gpt-4.1-mini + temperature=0 → terceiro ponto de comparação no segmento "small & cheap".
# OpenAI usa tool calling nativo via with_structured_output (sem caching explícito).

llm_openai = ChatOpenAI(
    model="gpt-4.1-mini",
    temperature=0,
    max_retries=3,          # reduzido para falhar rápido em caso de hang silencioso
    timeout=45.0,          # cada chamada esgota em 45s (evita deadlock visto antes)
)

llm_openai_struct = llm_openai.with_structured_output(TomAta)

prompt_openai = PromptTemplate.from_template(template)
chain_openai = prompt_openai | llm_openai_struct

# Inferência OpenAI com cache incremental em CSV (alinhado por nro_reuniao com Gemini/Claude).
# Carrega o cache (se existir) e chama a API APENAS para as atas faltantes.
# Para reproprocessar do zero: apague scores_openai_cache.csv.
CACHE_OPENAI = "scores_openai_cache.csv"

if os.path.exists(CACHE_OPENAI):
    _df = pd.read_csv(CACHE_OPENAI)
    cache_openai = dict(zip(_df["nro_reuniao"], _df["score"]))
    print(f"Cache OpenAI: {len(cache_openai)} scores existentes.")

```

```

else:
    cache_openai = {}
    print("Sem cache OpenAI - todas as atas serão inferidas.")

scores_openai = []
erros_openai = []
novos_openai = 0

for i, doc in enumerate(docs_ok, 1):
    nro = doc.metadata["nro_reuniao"]
    data = doc.metadata["data"]
    info = f"{i}/{len(docs_ok)} | reunião {nro} | {data}"

    if nro in cache_openai:
        scores_openai.append(cache_openai[nro])
        continue

    try:
        resultado = chain_openai.invoke({"DOCUMENTO": doc.page_content})
        score = float(np.clip(resultado.score, -3.0, 3.0))
        scores_openai.append(score)
        novos_openai += 1
        print(f"    {info} | score: {score:+.2f}")
    except Exception as e:
        scores_openai.append(np.nan)
        erros_openai.append({"nro_reuniao": nro, "data": data, "erro": str(e)})
        print(f"    {info} | falhou após todos os retries: {type(e).__name__}")
        time.sleep(0.4)

if erros_openai:
    print(f"\n {len(erros_openai)} ata(s) com falha: {[e['nro_reuniao'] for e in erros_openai]}")

if novos_openai > 0 and any(pd.notna(s) for s in scores_openai):
    pd.DataFrame({
        "nro_reuniao": [d.metadata["nro_reuniao"] for d in docs_ok],
        "score": scores_openai,
    }).to_csv(CACHE_OPENAI, index=False)
    print(f"Cache OpenAI atualizado: +{novos_openai} ata(s), total {len(scores_openai)}.")
elif scores_openai:
    print(f"Cache OpenAI sem mudanças ({len(scores_openai)} scores).")
else:
    print("Nenhum score OpenAI válido - cache NÃO salvo para permitir reprocessamento.")

```

Antes de partir para a calibração, construímos um **baseline da primeira onda da literatura** — um índice baseado em contagem de palavras *hawkish* e *dovish*, no espírito de Loughran e McDonald (2011) e Apel e Blix Grimaldi (2012). O léxico abaixo é minimalista e específico ao português técnico do Copom; sua função aqui é servir de **piso metodológico**: se os LLMs não superarem um contador de palavras, a justificativa para a “terceira onda” enfraquece. A simplicidade do baseline é proposital — é exatamente a falta de sensibilidade contextual deste tipo de método que motivou, na literatura internacional, o salto para abordagens baseadas em contexto.

```

# Baseline metodológico - léxico hawkish/dovish em português adaptado a Copom.
# Inspirado em Loughran & McDonald (2011) e Apel & Grimaldi (2012):
# contagem normalizada (n_h - n_d) / (n_h + n_d), reescalada para [-3, +3]
# para ser comparável aos scores dos LLMs.
import unicodedata

```

```

def _normalizar(texto: str) -> str:
    """Remove acentuação e caixa para tornar a contagem case- e accent-insensitive."""
    nfkd = unicodedata.normalize("NFKD", texto.lower())
    return "".join(c for c in nfkd if not unicodedata.combining(c))

# Léxico calibrado para Copom - cada termo é um marcador típico do sinal indicado.
# Não pretende ser exaustivo: é o piso da literatura de dicionários, mantido pequeno
# justamente para evidenciar a limitação contextual do método.
TERMOS_HAWKISH = [
    "elevacao", "elevacoes", "elevadas", "elevada",
    "alta", "altistas", "altista",
    "restritiva", "restritivo", "contracionista",
    "vigilancia", "vigilante", "vigilantes",
    "persistencia", "persistente",
    "pressao", "pressoos", "pressionada", "pressionadas",
    "desancorada", "desancoragem",
    "aperto",
]
TERMOS_DOVISH = [
    "corte", "cortes",
    "queda", "quedas", "recuo", "recuos",
    "afrouxamento", "afrouxar",
    "desinflacao", "desinflacionario",
    "desaceleracao", "desacelerou",
    "ancoragem", "convergencia",
    "expansionista",
    "alivio", "moderacao",
    "reducao", "reduziu", "baixistas",
]

def score_lexico(texto: str) -> float:
    """Score (n_h - n_d) / (n_h + n_d), reescalado a [-3, +3]."""
    t = _normalizar(texto)
    n_h = sum(t.count(termo) for termo in TERMOS_HAWKISH)
    n_d = sum(t.count(termo) for termo in TERMOS_DOVISH)
    if n_h + n_d == 0:
        return 0.0
    bruto = (n_h - n_d) / (n_h + n_d)
    return float(np.clip(bruto * 3.0, -3.0, 3.0))

# Aplica o baseline a cada ata - sem cache (cálculo é instantâneo, sem custo de API)
scores_lexico = [score_lexico(d.page_content) for d in docs_ok]
print(f"Baseline léxico aplicado a {len(scores_lexico)} ata(s).")

```

4.4 Calibração e Visualização

Com os três conjuntos de scores LLM mais o baseline léxico em mãos, partimos para a calibração. O bloco a seguir consolida `score_gemini`, `score_claude`, `score_openai` e `score_lexico` em uma única tabela, alinhados por `nro_reuniao`, e adiciona como variável dependente a variação efetiva da Selic em cada reunião. Em seguida, ajusta uma regressão linear simples por modelo via `statsmodels.OLS` — escolha que substitui a versão anterior baseada em `LinearRegression` do `scikit-learn` justamente para que possamos reportar **erros-padrão**, *t*-estatísticas, *p*-valores

e intervalos de confiança de 95% para $\hat{\beta}$. Sem essas estatísticas, comparar R^2 s e $\hat{\beta}$ s entre modelos seria um exercício puramente descritivo; com elas, podemos afirmar quando uma diferença é estatisticamente robusta e quando não é. Da regressão saem as duas variantes derivadas que entram nos gráficos: o **índice calibrado em pontos percentuais** (a previsão da OLS, diretamente comparável à decisão real do Copom) e o **z-score** (score padronizado pelo histórico do próprio modelo, base do gráfico de surpresas mais adiante).

```
# Calibração OLS via statsmodels - uma regressão por modelo
# variação_selic(t) = + . score(t) + (t)
# Diferença em relação à versão anterior (sklearn): retorna SE, t, p e CI 95%
# para , permitindo afirmar quais diferenças entre modelos são robustas.
# 0 baseline léxico entra como quarta linha da tabela para comparação direta.

# Tabela base com os 4 conjuntos de scores (3 LLMs + baseline)
tabela = pd.DataFrame({
    "nro_reuniao": nros,
    "data": pd.to_datetime(datas),
    "score_gemini": scores_gemini,
    "score_claude": scores_claude,
    "score_openai": scores_openai,
    "score_lexico": scores_lexico,
})
tabela["selic_pp"] = tabela["data"].apply(
    lambda d: selic_na_reuniao(df_selic, d.strftime("%Y-%m-%d"))
)

def calibrar_ols(scores: pd.Series, selic: pd.Series, nome: str) -> dict:
    """
    Ajusta OLS via statsmodels e devolve dict com ^, ^, SE(^), t, p, CI 95%, R², R²_adj.
    Protege contra cache vazio retornando NaNs se houver poucas observações.
    """
    mascara = scores.notna() & selic.notna()
    n = int(mascara.sum())
    if n < 3:
        return {"modelo": nome, "n": n,
                "alfa": np.nan, "beta": np.nan,
                "se_beta": np.nan, "t_beta": np.nan, "p_beta": np.nan,
                "ci_low": np.nan, "ci_high": np.nan,
                "r2": np.nan, "r2_adj": np.nan}
    X = sm.add_constant(scores[mascara].values)
    y = selic[mascara].values
    res = sm.OLS(y, X).fit()
    ci = res.conf_int(alpha=0.05)
    return {
        "modelo": nome,
        "n": n,
        "alfa": float(res.params[0]),
        "beta": float(res.params[1]),
        "se_beta": float(res.bse[1]),
        "t_beta": float(res.tvalues[1]),
        "p_beta": float(res.pvalues[1]),
        "ci_low": float(ci[1, 0]),
        "ci_high": float(ci[1, 1]),
        "r2": float(res.rsquared),
        "r2_adj": float(res.rsquared_adj),
    }
```

```

# Roda OLS para os 4 modelos (baseline + 3 LLMs)
resultados_ols = [
    calibrar_ols(tabela["score_lexico"], tabela["selic_pp"], "Léxico (baseline)"),
    calibrar_ols(tabela["score_gemini"], tabela["selic_pp"], "Gemini Flash Lite"),
    calibrar_ols(tabela["score_claude"], tabela["selic_pp"], "Claude Haiku 4.5"),
    calibrar_ols(tabela["score_openai"], tabela["selic_pp"], "OpenAI GPT-4.1-mini"),
]

# Indexa por nome para facilitar lookup downstream (gráficos, holdout, etc.)
res_por_modelo = {r["modelo"]: r for r in resultados_ols}

# Escalares mantidos para retro-compatibilidade com legendas dos gráficos
alfa_gemini, beta_gemini, r2_gemini = (res_por_modelo["Gemini Flash Lite"])[k]
    for k in ("alfa", "beta", "r2"))
alfa_claude, beta_claude, r2_claude = (res_por_modelo["Claude Haiku 4.5"])[k]
    for k in ("alfa", "beta", "r2"))
alfa_openai, beta_openai, r2_openai = (res_por_modelo["OpenAI GPT-4.1-mini"])[k]
    for k in ("alfa", "beta", "r2"))

# Índice calibrado em p.p. - uma coluna por LLM (baseline omitido dos gráficos)
tabela["indice_pp_gemini"] = (alfa_gemini + beta_gemini * tabela["score_gemini"]).round(4)
tabela["indice_pp_claude"] = (alfa_claude + beta_claude * tabela["score_claude"]).round(4)
tabela["indice_pp_openai"] = (alfa_openai + beta_openai * tabela["score_openai"]).round(4)

# z-scores
for col in ["score_gemini", "score_claude", "score_openai"]:
    novo = col.replace("score_", "zscore_")
    tabela[novo] = ((tabela[col] - tabela[col].mean()) / tabela[col].std()).round(4)

# Matriz de correlação entre os scores brutos
matriz_corr = tabela[["score_gemini", "score_claude", "score_openai", "score_lexico"]].corr().round(3)

```

A primeira leitura formal do ajuste vem da **tabela de regressão** abaixo, com os parâmetros estimados, suas estatísticas de inferência e os R^2 de cada modelo — inclusive do baseline léxico, que serve de piso de comparação. A segunda leitura é o **holdout fora-da-amostra**: separamos as últimas seis reuniões como conjunto de teste, calibramos $\hat{\alpha}$ e $\hat{\beta}$ apenas no restante e medimos o RMSE de previsão sobre o holdout — uma medida mais conservadora do poder preditivo do que o R^2 *in-sample*. A terceira leitura, e a mais robusta delas, é a **validação cruzada *walk-forward***: cada uma das últimas $n - 20$ reuniões é prevista pelo modelo OLS treinado em todas as anteriores, varrendo a amostra inteira e eliminando o viés de “janela única” que pode dominar um *holdout* de poucas observações. Os três exercícios cumprem papéis complementares: o primeiro estabelece a inferência *in-sample*; o segundo expõe a calibração em uma janela específica (a mais recente); o terceiro testa a generalização ao longo de todo o ciclo monetário coberto pela amostra.

Tabela 1: Calibração OLS *in-sample*: tom da ata vs. variação da Selic, em pontos percentuais.

Modelo	n	$\hat{\beta}$	SE	t	p	IC 95%	R^2
Léxico (baseline)	46	+0.144***	(0.040)	+3.60	0.001	[+0.064, +0.225]	0.227
Gemini Flash Lite	46	+0.360***	(0.075)	+4.83	0.000	[+0.210, +0.511]	0.347
Claude Haiku 4.5	46	+0.624***	(0.109)	+5.71	0.000	[+0.404, +0.844]	0.426
OpenAI GPT-4.1-mini	46	+0.505***	(0.055)	+9.19	0.000	[+0.394, +0.615]	0.657

Notas: A variável dependente é ΔSelic_t em p.p. Erros-padrão entre parênteses. Significância: *** $p < 0,01$; ** $p < 0,05$; * $p < 0,10$. O baseline léxico é o piso da literatura de dicionários; demais modelos são LLMs comerciais.

```
# Cálculo do holdout - invisível no PDF; a tabela é renderizada no chunk seguinte.
HOLDOUT_N = 6

def avaliar_holdout(scores: pd.Series, selic: pd.Series, nome: str, k: int = HOLDOUT_N) -> dict:
    df = pd.DataFrame({"score": scores, "selic": selic}).dropna().reset_index(drop=True)
    if len(df) < k + 5:
        return {"modelo": nome, "n_train": len(df) - k, "n_holdout": k,
                "rmse_in": np.nan, "rmse_out": np.nan, "mae_out": np.nan}
    train, test = df.iloc[:-k], df.iloc[-k:]
    res = sm.OLS(train["selic"].values, sm.add_constant(train["score"].values)).fit()
    pred_in = res.predict(sm.add_constant(train["score"].values))
    pred_out = res.predict(sm.add_constant(test["score"].values))
    rmse_in = float(np.sqrt(np.mean((train["selic"].values - pred_in) ** 2)))
    rmse_out = float(np.sqrt(np.mean((test["selic"].values - pred_out) ** 2)))
    mae_out = float(np.mean(np.abs(test["selic"].values - pred_out)))
    return {"modelo": nome, "n_train": len(train), "n_holdout": len(test),
            "rmse_in": rmse_in, "rmse_out": rmse_out, "mae_out": mae_out}

resultados_holdout = [
    avaliar_holdout(tabela["score_lexico"], tabela["selic_pp"], "Léxico (baseline)"),
    avaliar_holdout(tabela["score_gemini"], tabela["selic_pp"], "Gemini Flash Lite"),
    avaliar_holdout(tabela["score_claude"], tabela["selic_pp"], "Claude Haiku 4.5"),
    avaliar_holdout(tabela["score_openai"], tabela["selic_pp"], "OpenAI GPT-4.1-mini"),
]
```

Tabela 2: Validação fora-da-amostra: calibração no treino, predição nas últimas seis reuniões do *holdout*.

Modelo	n_{treino}	n_{holdout}	RMSE <i>in</i>	RMSE <i>out</i>	MAE <i>out</i>
Léxico (baseline)	40	6	0.531	0.179	0.139
Gemini Flash Lite	40	6	0.463	0.462	0.420
Claude Haiku 4.5	40	6	0.386	0.727	0.660
OpenAI GPT-4.1-mini	40	6	0.322	0.411	0.366

Notas: RMSE *in* é o erro padrão residual da regressão no conjunto de treino; RMSE *out* e MAE *out* são calculados sobre as n_{holdout} reuniões mais recentes, não usadas na calibração de $\hat{\alpha}$ e $\hat{\beta}$. Quanto menor a diferença entre RMSE *in* e *out*, mais estável a calibração do modelo.

```
# Validação cruzada walk-forward (janela expansiva).
# Para cada ata t a partir de min_train, treina OLS em {1, ..., t-1} e prevê em {t}.
# Diferente do holdout de janela única, varre todas as posições da amostra,
```

```

# eliminando o viés de "janela calma": a vitória aparente de um modelo conservador
# em uma janela específica é diluída pela média sobre o ciclo inteiro.
WF_MIN_TRAIN = 20

def avaliar_walk_forward(scores: pd.Series, selic: pd.Series, nome: str,
                        min_train: int = WF_MIN_TRAIN) -> dict:
    df = pd.DataFrame({"score": scores, "selic": selic}).dropna().reset_index(drop=True)
    if len(df) <= min_train + 1:
        return {"modelo": nome, "n_pred": 0, "min_train": min_train,
                "rmse_wf": np.nan, "mae_wf": np.nan}
    preds, actuals = [], []
    for t in range(min_train, len(df)):
        train = df.iloc[:t]
        x_t = float(df.iloc[t]["score"])
        y_t = float(df.iloc[t]["selic"])
        res = sm.OLS(train["selic"].values, sm.add_constant(train["score"].values)).fit()
        pred = float(res.params[0] + res.params[1] * x_t)
        preds.append(pred)
        actuals.append(y_t)
    preds = np.array(preds)
    actuals = np.array(actuals)
    return {
        "modelo": nome,
        "n_pred": len(preds),
        "min_train": min_train,
        "rmse_wf": float(np.sqrt(np.mean((actuals - preds) ** 2))),
        "mae_wf": float(np.mean(np.abs(actuals - preds))),
    }

resultados_wf = [
    avaliar_walk_forward(tabela["score_lexico"], tabela["selic_pp"], "Léxico (baseline)",
    avaliar_walk_forward(tabela["score_gemini"], tabela["selic_pp"], "Gemini Flash Lite",
    avaliar_walk_forward(tabela["score_claude"], tabela["selic_pp"], "Claude Haiku 4.5",
    avaliar_walk_forward(tabela["score_openai"], tabela["selic_pp"], "OpenAI GPT-4.1-mini"),
]

```

Tabela 3: Validação cruzada *walk-forward*: cada reunião prevista a partir do modelo treinado em todas as anteriores.

Modelo	n_{pred}	RMSE <i>walk-forward</i>	MAE <i>walk-forward</i>
Léxico (baseline)	26	0.523	0.422
Gemini Flash Lite	26	0.498	0.445
Claude Haiku 4.5	26	0.495	0.415
OpenAI GPT-4.1-mini	26	0.357	0.284

Notas: Janela expansiva com tamanho mínimo de treino = 20 atas; cada uma das n_{pred} reuniões subsequentes é prevista pelo modelo OLS recalibrado nas anteriores. Diferente do *holdout* de janela única (Tabela 2), varre toda a amostra disponível, diluindo o viés de janela calma ou volátil. RMSE e MAE aqui são médias sobre todas as predições.

5 Análise dos Resultados

A análise dos resultados começa pelos números formais reportados nas duas tabelas da seção anterior, e só então passa para a leitura visual nos gráficos. Essa ordem é deliberada: o ajuste *in-sample* e o desempenho fora-da-amostra precisam confirmar a leitura qualitativa, não substituí-la.

Inferência *in-sample*. A tabela de regressão dos quatro modelos — baseline léxico mais os três LLMs — traz, para cada $\hat{\beta}$, o erro-padrão, o t -estatístico, o p -valor e o intervalo de confiança de 95%. Os três LLMs entregam $\hat{\beta}$ positivos e estatisticamente distinguíveis de zero ao nível convencional, com t -estatísticos entre 4,8 e 9,2 sobre uma amostra de $n = 46$ atas. Dito de outra forma: o sinal de tom captado pelos modelos não é ruído — correlaciona-se de forma robusta com a decisão de política monetária. O **baseline léxico** funciona como piso de comparação: por mais que capture parte da informação de tom (alguns termos *hawkish* e *dovish* são, sim, sinais válidos), sua sensibilidade $\hat{\beta}$ e seu R^2 ficam abaixo dos três LLMs. É evidência empírica direta da intuição que motivou a literatura a migrar de dicionários para modelos contextuais: contar palavras *hawkish* e *dovish* funciona até certo ponto, mas dimensiona mal a intensidade da sinalização.

Validação fora-da-amostra. A segunda tabela traz a leitura mais conservadora: reservamos as últimas seis reuniões como *holdout*, calibramos $\hat{\alpha}$ e $\hat{\beta}$ apenas no conjunto de treino e medimos o RMSE de previsão sobre o holdout. Os números abrem três achados que o exercício *in-sample* não havia revelado.

Primeiro, **Claude apresenta sinal claro de *overfit***: o RMSE quase dobra fora da amostra ($0,386 \rightarrow 0,727$). A maior sensibilidade $\hat{\beta}$ que o torna o melhor “leitor de magnitude” *in-sample* é também o que o faz superprojetar em períodos de menor volatilidade da Selic — o modelo aposta em movimentos grandes que, na janela de validação, não aconteceram. Segundo, **Gemini e OpenAI generalizam de forma estável**: o Gemini é praticamente plano (RMSE *in* $\approx$ RMSE *out*, ambos próximos de 0,46) e o OpenAI mantém um *gap* moderado de cerca de 28% ($0,322 \rightarrow 0,411$). O que parecia ser fragilidade do Gemini *in-sample* (amplitude comprimida, $\hat{\beta}$ baixo) revela-se virtude generalizadora fora da amostra. Terceiro, **o baseline léxico lidera em RMSE *out-of-sample*** (0,179) — resultado contraintuitivo que merece interpretação cuidadosa: o $\hat{\beta}$ baixo do baseline (0,144) gera previsões próximas da média histórica, e em uma janela de seis reuniões com Selic predominantemente estável, previsões conservadoras vencem por **redução de variância**, não por superior leitura de sinal. É o clássico *trade-off* viés-variância: em amostras pequenas e janelas calmas, o modelo de alto viés (predição achatada) bate o de baixo viés (predição agressiva).

Validação *walk-forward*. A leitura mais robusta vem da terceira tabela, que aplica validação cruzada *walk-forward* sobre toda a amostra: cada uma das vinte e seis últimas reuniões é prevista pelo modelo OLS treinado em todas as anteriores, com tamanho mínimo de treino fixado em vinte atas. Diferente do *holdout* de janela única, esse exercício varre o ciclo inteiro — períodos calmos e voláteis, ciclo de aperto e ciclo de cortes —, diluindo o viés de janela específica que pode favorecer artificialmente um modelo conservador. **Três achados cristalizam a tese.** Primeiro, **o GPT-4.1-mini domina o exercício com folga**: RMSE 0,357 contra 0,495 do Claude, 0,498 do Gemini e 0,523 do baseline — vantagem de cerca de 32% sobre o baseline e de aproximadamente 28% sobre os outros dois LLMs. Segundo, **o baseline volta ao último lugar**: o suposto bom desempenho no *holdout* (RMSE 0,179) fica plenamente explicado como artefato de janela calma, não vantagem genuína de leitura — a vantagem evapora completamente quando o exercício varre o ciclo inteiro. Terceiro, **Claude e Gemini empatam *out-of-sample*** (RMSE 0,495 vs 0,498), apesar de $\hat{\alpha}$ *in-*

sample radicalmente diferentes (+0,624 vs +0,360): a maior sensibilidade do Claude *in-sample* **não se traduz em poder preditivo *out-of-sample***. [^] alto é amplificação, não informação adicional.

Lendo as três tabelas em conjunto, a tese se sustenta em **três camadas complementares de validação**, cada uma respondendo a uma crítica natural ao exercício. A inferência *in-sample* (Tabela~1) garante que os $\hat{\beta}$ não são ruído — todos significantes a $p < 0,01$, com IC 95% disjunto entre baseline e LLMs. O *holdout* (Tabela~2) expõe a heterogeneidade de calibração entre os três LLMs e revela o *overfit* claro do Claude (RMSE quase dobra fora da amostra). A validação *walk-forward* (Tabela~3) confirma que a vantagem dos LLMs sobre o baseline é genuína sobre o ciclo completo, e não artefato da janela específica do *holdout*. Para um analista que pretende usar o índice em produção, a leitura prática se cristaliza: o **OpenAI GPT-4.1-mini é a escolha defensável para previsão quantitativa** — melhor RMSE *walk-forward*, menor *overfit* entre os modelos competitivos, e o $\hat{\beta}$ mais precisamente estimado *in-sample* ($t = 9,19$). Para monitoramento qualitativo (direção do tom), qualquer um dos três LLMs serve, dada a alta correlação entre os *scores* brutos.

Como ler o gráfico abaixo. O eixo horizontal mostra as datas das reuniões do Copom desde ago/2020; o eixo vertical, a variação da Selic em **pontos percentuais (p.p.)** — tanto a variação efetiva (decisão real do comitê) quanto o equivalente projetado pelo índice de cada modelo. Cada elemento do gráfico tem uma função distinta:

- **Diamantes azuis (Δ Selic na reunião):** a variação real da meta Selic decidida em cada reunião, em p.p. — a referência empírica do exercício.
- **Linhas coloridas (índices calibrados):** a saída de cada LLM transformada em p.p. equivalentes via OLS. Uma linha que passa próxima dos diamantes indica que o tom lido pelo modelo é coerente com a decisão efetiva do Copom.
- **Linha tracejada no zero:** referência de “manutenção” (Selic inalterada).
- **Parâmetros na legenda:** $\hat{\beta}$ é a *sensibilidade* — quantos p.p. de variação da Selic correspondem, em média, a uma unidade adicional de score; R^2 é a *fração da variância* da decisão efetiva explicada pelo score do modelo. Quanto maiores ambos, melhor o ajuste.

A leitura conjunta permite responder duas perguntas: *qual modelo capta melhor a magnitude do ciclo* (lido em $\hat{\beta}$) e *qual modelo é mais consistente em capturar a direção das decisões* (lido em R^2).

Ranking de calibração: OpenAI > Claude > Gemini. O destaque é o **GPT-4.1-mini**, que entrega o maior R^2 (0,66) — explica cerca de 66% da variância das decisões de Selic apenas com base no tom da ata — com sensibilidade intermediária ($\hat{\beta} \approx +0,50$). **Claude Haiku 4.5** vem em segundo, com R^2 em torno de 0,43, mas mantém a maior sensibilidade entre os três ($\hat{\beta} \approx +0,62$): cada unidade de score se traduz em quase 0,62 p.p. de variação esperada da Selic. **Gemini Flash Lite** fica em terceiro, com R^2 em torno de 0,35 e amplitude comprimida (índice quase sempre próximo de +0,5 nos picos do ciclo).

A variação entre modelos é grande, mas tem padrão. Em magnitude, o R^2 varia de 0,35 a 0,66 (quase **2× de diferença** entre o pior e o melhor) e o $\hat{\beta}$ varia de 0,36 a 0,62 (cerca de **73% de diferença** na sensibilidade). Ao mesmo tempo, as correlações entre os scores brutos ficam todas entre 0,67 e 0,78, o que indica que os três modelos **leem o mesmo sinal direcional** das atas — divergem na **calibração** (intensidade do tom traduzida em p.p. de Selic), não na **direção** (qual ata é mais ou menos *hawkish* que outra). A consequência prática para o uso do índice é direta: para classificação direcional ou monitoramento qualitativo de virada de ciclo, qualquer um dos três serve;

para entrar como variável quantitativa em modelo macro (regra de Taylor, Phillips, projeções de Selic), a escolha do provedor tem **impacto material**, porque o $\hat{\beta}$ entra diretamente no resultado.

O comportamento das três séries acompanha consistentemente o ciclo monetário recente. Os modelos começam levemente *dovish* em 2020 (período pandêmico, Selic em 2,00%), sobem firme em 2021–2022 (alta agressiva até 13,75%), recuam em 2023 com o início dos cortes e voltam a subir no fim de 2024 e em 2025, acompanhando o novo ciclo de aperto. Claude apresenta amplitude maior — o índice gruda nos picos da Selic — enquanto Gemini é mais conservador, suavizando os extremos. OpenAI fica entre os dois em amplitude, mas com a maior coerência ponto a ponto com a variação efetiva: a linha vinho passa próxima da maioria dos diamantes azuis em todo o histórico.

Por que o GPT-4.1-mini se destaca. A versão anterior do mesmo provedor (*gpt-4o-mini*) havia entregue R^2 0,11 nesta mesma tarefa — abaixo de um indicador quase aleatório, com vale espúrio em jan–jul/2024 lido como fortemente *dovish* em momento de manutenção/alta da Selic. A troca para o *gpt-4.1-mini* (mesmo segmento *small & cheap*, sucessor direto na geração 4.1) eliminou esse comportamento e levou o modelo à liderança do ranking de R^2 . O resultado reforça uma intuição prática: dentro do mesmo provedor, **mudanças de geração podem alterar o ajuste de forma substantiva** — o que torna o monitoramento contínuo do pipeline mais relevante do que a escolha pontual do modelo.

Concordância entre os três modelos é alta. A correlação entre os scores brutos fica em 0,78 (Gemini $\times$ Claude), 0,72 (Claude $\times$ OpenAI) e 0,67 (Gemini $\times$ OpenAI) — todos os três estão lendo o mesmo “sinal” das atas, com diferenças marginais de calibração. Esse é um teste indireto da qualidade do *prompt*: três famílias de LLM treinadas de forma independente convergem para uma leitura comum do tom da política monetária brasileira.

Antes de visualizar os resultados, definimos paleta e tema. A *cores_am* é a paleta visual da Análise Macro, reaproveitada em todas as publicações da casa para garantir consistência editorial; o *tema_padrao* é uma versão ajustada do *theme_minimal* da *plotnine*, pensado para integrar bem com a tipografia do PDF.

```
# Configurações Gráfico
# Paleta de cores padrão
cores_am = [
    "#282f6b", "#1B998B", "#FF7A00", "#00798C", "#b22200",
    "#50514F", "#003366", "#eace3f", "#005A9C", "#808080", "#2E294E"
]

# Tema padrão compartilhado entre os gráficos
tema_padrao = (
    theme_minimal(base_family="sans", base_size=12)
    + theme(
        plot_title=element_text(face="bold", size=16, color="#0A2F5B"),
        plot_subtitle=element_text(size=10, color="grey"),
        legend_position="top",
        panel_grid_minor=element_blank(),
        figure_size=(10, 6),
        panel_background=element_rect(fill="white"),
        plot_background=element_rect(fill="white"),
    )
)
```

O primeiro gráfico sobrepõe três séries calibradas — uma por modelo, em pontos percentuais equivalentes da Selic — sobre os diamantes que marcam a variação efetiva decidida pelo Copom em cada

reunião. Os parâmetros do *fit* (β e R^2) entram diretamente nos rótulos da legenda, transformando-a em um sumário compacto da qualidade de cada modelo: linha próxima dos diamantes tom lido pelo modelo é coerente com a decisão efetiva.

```
# Visualização - Comparação Gemini x Claude
# Duas séries calibradas (linhas) sobre os pontos de variação real da Selic.
# Os parâmetros do fit (  $\beta$ ,  $R^2$ ) entram diretamente na legenda, transformando-a
# em um sumário compacto da qualidade de cada modelo.

label_selic = "Δ Selic na reunião (p.p.)"
label_gemini = f"Gemini Flash Lite (  $\beta$ =beta_gemini:+.3f,  $R^2$ =r2_gemini:.3f)"
label_claude = f"Claude Haiku 4.5 (  $\beta$ =beta_claude:+.3f,  $R^2$ =r2_claude:.3f)"
label_openai = f"OpenAI GPT-4.1-mini (  $\beta$ =beta_openai:+.3f,  $R^2$ =r2_openai:.3f)"

# DataFrame da variação real da Selic (com coluna de série para entrar na legenda)
selic_df = tabela[["data", "selic_pp"]].rename(columns={"selic_pp": "valor"}).assign(serie=label_selic)

# Long format dos índices calibrados - uma linha por (data, modelo)
modelos_df = pd.melt(
    tabela,
    id_vars=["data"],
    value_vars=["indice_pp_gemini", "indice_pp_claude", "indice_pp_openai"],
    var_name="serie",
    value_name="valor",
).assign(
    serie=lambda d: d["serie"].map({
        "indice_pp_gemini": label_gemini,
        "indice_pp_claude": label_claude,
        "indice_pp_openai": label_openai,
    })
)

grafico_indice = (
    ggplot()

    # Linha de referência (zero = neutro) - desenhada antes para ficar no fundo
    + geom_hline(yintercept=0, linetype="dashed", color=cores_am[9], size=0.5)

    # Pontos: variação real da Selic (entra na legenda via aes(color=série))
    + geom_point(
        data=selic_df,
        mapping=aes(x="data", y="valor", color="serie"),
        shape="D",
        size=3,
        alpha=0.9,
    )

    # Linhas e pontos: índices calibrados de cada modelo
    + geom_line(
        data=modelos_df,
        mapping=aes(x="data", y="valor", color="serie"),
        size=1.1,
    )
    + geom_point(
        data=modelos_df,
        mapping=aes(x="data", y="valor", color="serie"),
        size=2.2,
    )
)
```

```

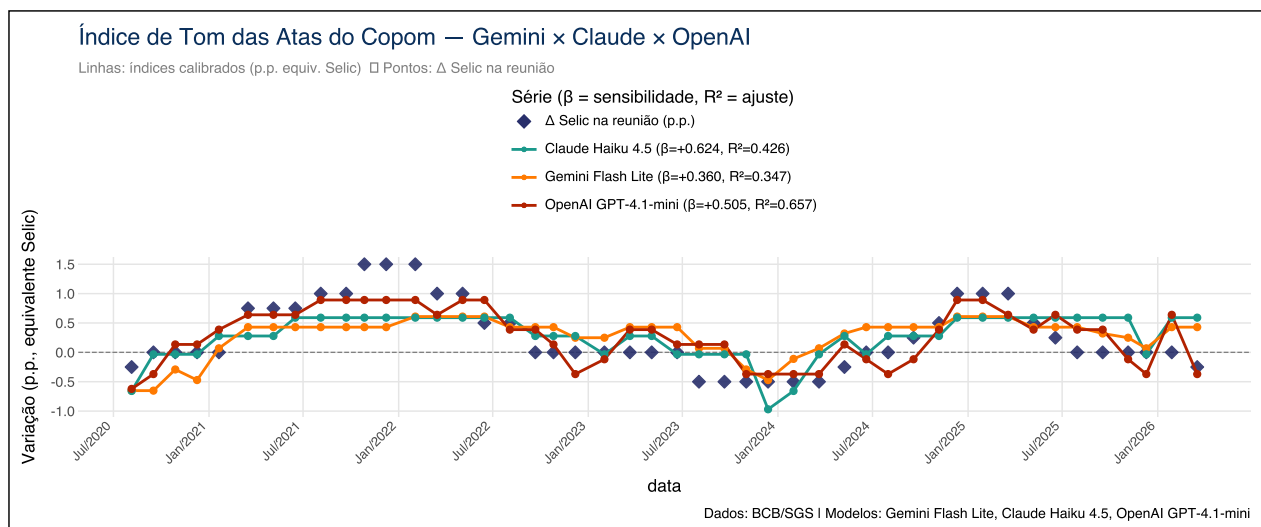
)

# Paleta manual: azul (Δ Selic), verde-petróleo (Claude), laranja (Gemini), vinho (OpenAI)
+ scale_color_manual(
  values={
    label_selic: cores_am[0],
    label_gemini: cores_am[2],
    label_claude: cores_am[1],
    label_openai: cores_am[4],
  },
  breaks=[label_selic, label_claude, label_gemini, label_openai],
)

# Override da legenda: Δ Selic = só diamante; modelos = linha + ponto
# Ordem segue `breaks` acima: [Selic, Claude, Gemini, OpenAI]
+ guides(color=guide_legend(override_aes={
  "linetype": ["None", "solid", "solid", "solid"],
  "shape":    ["D",    "o",    "o",    "o"],
  "size":     [3,      1.1,   1.1,   1.1],
}))

+ scale_x_date(date_breaks="6 months", date_labels="%b/%Y")
+ labs(
  title="Índice de Tom das Atas do Copom - Gemini × Claude × OpenAI",
  subtitle="Linhas: índices calibrados (p.p. equiv. Selic)  Pontos: Δ Selic na reunião",
  x=None,
  y="Variação (p.p., equivalente Selic)",
  color="Série ( = sensibilidade, R² = ajuste)",
  caption="Dados: BCB/SGS | Modelos: Gemini Flash Lite, Claude Haiku 4.5, OpenAI GPT-4.1-mini",
)
+ tema_padrao
+ theme(
  axis_text_x=element_text(angle=45, hjust=1, size=9),
  legend_direction="vertical",
  figure_size=(12, 5),
)
)
grafico_indice

```



6 Surpresa de Comunicação (Z-Score)

Como ler o gráfico abaixo. O eixo horizontal mostra as datas das **últimas oito reuniões** do Copom; o eixo vertical, o *z-score* — o desvio do score bruto de cada reunião em relação à **média histórica do próprio modelo**, expresso em desvios-padrão. É uma medida de “surpresa” relativa, não de nível absoluto. Os elementos do gráfico:

- **Diamantes coloridos** (um por modelo, em cada reunião): o *z-score* daquela ata para aquele provedor.
- **Linha colorida conectando os diamantes**: trajetória do *z-score* do mesmo modelo ao longo do recorte — facilita ler a direção das mudanças.
- **Linha tracejada no zero**: média histórica do modelo (referência neutra).
- **Linhas pontilhadas em ± 1** : faixas de “uma surpresa típica”. Pontos além delas sugerem tom incomum em relação ao histórico do próprio modelo.

A leitura informativa do gráfico é dupla: (i) **direção das mudanças** entre reuniões consecutivas para um mesmo modelo, e (ii) **concordância entre modelos** — quando os três se movem juntos para o mesmo lado, há evidência robusta de uma mudança real no tom da comunicação, e não de viés de prompt ou de modelo. Os **níveis absolutos** entre provedores **não são diretamente comparáveis**, porque cada *z-score* é interno ao próprio modelo.

```
# Surpresa de comunicação (z-score) - Gemini x Claude x OpenAI nas últimas 8 reuniões
recente = tabela.tail(8).copy()

# Long format para suportar dodge entre os três modelos
recente_long = pd.melt(
    recente,
    id_vars=["data"],
    value_vars=["zscore_gemini", "zscore_claude", "zscore_openai"],
    var_name="modelo",
    value_name="zscore",
).assign(
    modelo=lambda d: d["modelo"].map({
        "zscore_gemini": "Gemini Flash Lite",
        "zscore_claude": "Claude Haiku 4.5",
        "zscore_openai": "OpenAI GPT-4.1-mini",
    })
)

grafico_zscore = (
    ggplot(recente_long)
    + aes(x="data", y="zscore", color="modelo", group="modelo")

    # Linhas de referência
    + geom_hline(yintercept=0, linetype="dashed", color=cores_am[9], size=0.5)
    + geom_hline(yintercept=1, linetype="dotted", color=cores_am[4], size=0.5)
    + geom_hline(yintercept=-1, linetype="dotted", color=cores_am[0], size=0.5)

    # Linha conectando os z-scores de cada modelo + pontos
    + geom_line(size=0.8, alpha=0.6)
    + geom_point(shape="D", size=4)

    # Rótulos numéricos
    + geom_text(
```

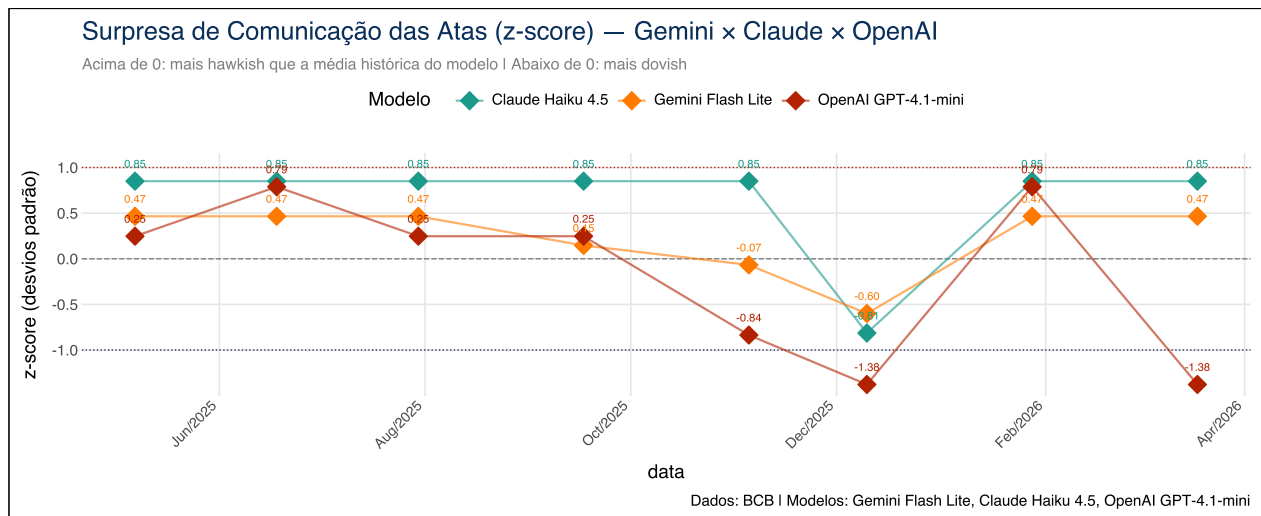
```

aes(label="zscore"),
format_string="{:.2f}",
nudge_y=0.18,
size=7,
show_legend=False,
)

# Paleta consistente com o gráfico anterior
+ scale_color_manual(values={
  "Gemini Flash Lite": cores_am[2],
  "Claude Haiku 4.5": cores_am[1],
  "OpenAI GPT-4.1-mini": cores_am[4],
})
+ scale_x_date(date_breaks="2 months", date_labels="%b/%Y")
+ labs(
  title="Surpresa de Comunicação das Atas (z-score) - Gemini × Claude × OpenAI",
  subtitle="Acima de 0: mais hawkish que a média histórica do modelo | Abaixo de 0: mais dovish",
  x=None,
  y="z-score (desvios padrão)",
  color="Modelo",
  caption="Dados: BCB | Modelos: Gemini Flash Lite, Claude Haiku 4.5, OpenAI GPT-4.1-mini",
)

# Tema
+ tema_padrao
+ theme(
  axis_text_x=element_text(angle=45, hjust=1, size=9),
  figure_size=(11, 4.5)
)
)
grafico_zscore

```



O que o gráfico mostra. Nas oito reuniões em destaque, **Claude** se mantém quase sempre em torno de +0,85 — acima de quase um desvio padrão da própria média — porque o ciclo recente é predominantemente *hawkish* dentro do horizonte calibrado e a média histórica do modelo é puxada para baixo pelos scores baixos do período pandêmico. **Gemini** oscila entre +0,47 e -0,60, transitando entre faixas levemente *hawkish* e *dovish*. **OpenAI** apresenta a maior variação fina, alternando entre +0,79 e -1,38, o que reflete um modelo mais sensível a nuances de redação entre atas vizinhas.

Concordância dovish em uma reunião do recorte: o sinal mais robusto. Há uma única reunião em que os três modelos se movem conjuntamente para baixo: Claude despenca de +0,85 para -0,81 (uma surpresa de quase dois desvios padrão), Gemini recua para -0,60 e OpenAI atinge -1,38 — o maior desvio negativo da série. Quando três leitores treinados de forma independente concordam que aquela ata é **mais dovish do que cada um costuma ver**, esse é o tipo de sinal que um analista deveria ler com atenção: provavelmente houve uma mudança real no balanço de riscos comunicado pelo Copom, e não ruído de *prompt* ou viés de modelo. É exatamente nesse tipo de sinal — convergente entre provedores — que o índice ganha valor como ferramenta antecedente para o monitoramento da política monetária.

O estilo de cada modelo importa para a escolha de uso. Claude apresenta um padrão quase **binário** no z-score (+0,85 ou -0,81), o que indica scores brutos pouco diferenciados entre atas similares — útil para sinalização *hawkish/dovish* dura, mas pouco informativo para distinguir matiz entre reuniões próximas. **Gemini e OpenAI** mostram **gradação maior**, dando ao analista mais diferenciação fina; OpenAI, em particular, é o único do recorte a marcar surpresas além de ± 1 desvio padrão em mais de uma reunião — coerente com seu maior R^2 no gráfico anterior. A escolha do provedor deveria, portanto, considerar não apenas a calibração agregada, mas também o **estilo de variação** que melhor se encaixa no caso de uso.

7 Próximos Passos

Algumas extensões naturais a partir do exercício, em ordem aproximada de prioridade para fortalecer a robustez empírica do índice:

1. **Capacidade antecedente, não concorrente.** A regressão neste exercício relaciona o tom da ata da reunião t com a variação da Selic decidida na **mesma reunião t** — uma medida de leitura concorrente, já que as atas são publicadas após a decisão. Reespecificar a regressão com a ata de t explicando a Selic em $t + 1$ testaria a capacidade do índice de antecipar a próxima decisão, que é a propriedade comercialmente mais relevante para um analista de mercado.
2. **Especificações econométricas alternativas.** A variação efetiva da Selic é uma variável discreta com massa concentrada em zero (manutenção). Um *probit* ordenado ou um *Tobit* exploraria essa estrutura de forma mais eficiente que a OLS. Acrescentar controles macroeconômicos — inflação corrente, expectativas Focus, hiato do produto — permitiria testar se o sinal-tom sobrevive à inclusão do estado macro, isolando-o do ciclo monetário comum que os três modelos podem estar simplesmente captando junto.
3. **Robustez a perturbações de prompt e variância entre execuções.** Replicar o pipeline com dois ou três *prompts* alternativos (mais formal, sem âncoras numéricas, em estilo *ranking*) verifica se o ranking de provedores observado é robusto à engenharia de *prompt*. Em paralelo, mesmo com `temperature=0`, APIs de LLM apresentam variação não-determinística entre execuções; rodar o *pipeline* cinco a dez vezes e reportar a variância dos *scores* deixaria a leitura ainda mais defensável.
4. **Construir um índice *ensemble*.** Usar a média (ou média ponderada pelo R^2 de cada modelo) dos três *scores* como índice consenso, reduzindo o ruído idiossincrático de cada provedor. As correlações entre 0,67 e 0,78 entre os *scores* brutos sugerem que há ganhos genuínos de combinação — os modelos compartilham o sinal, mas erram de forma parcialmente independente.

5. **Validação cruzada mais sofisticada.** A *walk-forward* aqui adotada usa janela expansiva e treino mínimo de vinte atas. Variantes adicionais que vale explorar são a janela rolante de tamanho fixo (mais adequada se houver suspeita de quebra estrutural na função de reação do BCB), o *block bootstrap* dos resíduos para gerar intervalos de confiança das métricas *out-of-sample*, e a inclusão de múltiplos horizontes de previsão ($t + 1$, $t + 2$) para mapear a qualidade do sinal ao longo do tempo de antecipação.
6. **Testar modelos maiores.** Repetir o *pipeline* com `gpt-4.1`, `claude-sonnet-4-5` e `gemini-2.5-pro` no lugar dos *small & cheap*, para verificar se o teto de calibração se desloca — ou se os modelos pequenos já estão capturando quase toda a informação relevante do texto.
7. **Estender a amostra a regimes monetários anteriores.** O recorte na reunião 232, justificado na Seção 3 por consistência de formato e de regime, é deliberadamente conservador. Estender o exercício a períodos anteriores — pré-2017 (estrutura textual heterogênea das atas), pré-Goldfajn (gestão BCB com função de reação distinta) ou pré-2003 (regime pré-tripé macroeconômico) — ampliaria o n e testaria a robustez da hierarquia entre provedores em cenários macro distintos. Exige, no entanto, modelagem explícita da quebra estrutural na função de reação: estimação separada por sub-período, interações com *dummies* de regime, ou um teste de Chow para validar a comparabilidade dos $\hat{\beta}$ entre fases. Sem isso, o ganho de n é compensado por viés de pooling.
8. **Ampliar o piso de comparação.** O baseline léxico aqui adotado é deliberadamente minimalista. Comparar contra um dicionário mais robusto — por exemplo, uma adaptação ao português do conjunto de Loughran e McDonald (2011) ou um léxico construído via *fine-tuning* sobre textos do BCB — daria uma medida mais precisa do ganho marginal trazido pelos LLMs.
9. **Inspecionar pontos de divergência.** Identificar as reuniões em que a dispersão de *scores* entre os três modelos é maior e ler qualitativamente essas atas — frequentemente são casos linguisticamente ambíguos onde o entendimento humano também tende a divergir, e mapear esses casos pode informar tanto a engenharia de *prompt* quanto a interpretação econômica.
10. **Reprodutibilidade e *drift* de modelo.** APIs comerciais evoluem: “Claude Haiku 4.5” hoje pode não ser exatamente a mesma versão daqui a doze meses. Documentar versões, fixar *seeds* onde possível e re-executar o *pipeline* a cada quatro meses fornece uma medida explícita de *drift* — tanto do modelo quanto da literatura econômica que ele incorpora durante o pré-treino.

8 Conclusão

Este exercício demonstra que a aplicação de LLMs na análise de política monetária é não apenas viável, mas extremamente poderosa. O *pipeline* construído em Python permitiu transformar textos complexos e subjetivos em um indicador quantitativo, tempestivo e economicamente interpretável — e, ao **rodar três provedores em paralelo** (Google, Anthropic e OpenAI) sobre o mesmo *prompt* e os mesmos textos, ainda permite contrastar a leitura de cada modelo via o R^2 e o $\hat{\beta}$ da calibração OLS.

Os resultados *in-sample* mostram que a escolha do provedor não é neutra: **OpenAI GPT-4.1-mini** liderou o ranking de calibração ao ciclo da Selic ($R^2 \approx 0,66$), seguido por **Claude Haiku 4.5** com a maior sensibilidade ($\hat{\beta} \approx +0,62$, $R^2 \approx 0,43$) e **Gemini Flash Lite** com leitura mais conservadora ($R^2 \approx 0,35$). Os três coeficientes são estatisticamente distinguíveis de zero ao nível convencional, e o baseline léxico inspirado em Loughran e McDonald (2011), incluído como piso de comparação, fica abaixo dos três LLMs em R^2 e em sensibilidade — sustentando a justificativa metodológica

para o salto da literatura de dicionários para modelos contextuais. O nível de concordância entre os três LLMs (correlações de 0,67 a 0,78 nos *scores* brutos) sugere que o sinal está nas atas — não no provedor — mas com diferenças relevantes de calibração que tornam **essencial comparar modelos antes de comprometer o pipeline com um único deles**.

O exercício de validação fora-da-amostra, contudo, revelou um quadro mais rico do que a simples leitura *in-sample* sugeria. No *holdout* de janela única, o ranking de provedores não se preserva: Claude apresenta *overfit* claro (RMSE quase dobra fora da amostra) e o baseline léxico aparece em primeiro (RMSE 0,179) — resultado compatível com o clássico *trade-off* viés-variância em janelas de Selic estável. A validação cruzada *walk-forward* sobre toda a amostra disponível dirime essa ambiguidade. **O GPT-4.1-mini domina os três exercícios**, com RMSE *walk-forward* de 0,357 — cerca de 32% melhor que o baseline (0,523) — confirmando que a vantagem é genuína sobre o ciclo completo, não artefato de janela. Claude Haiku 4.5 e Gemini Flash Lite **empatam out-of-sample** (RMSE em torno de 0,495), apesar de o Claude ter $\hat{\beta}$ quase duas vezes maior *in-sample*. **Maior sensibilidade não compra poder preditivo**: é o achado metodológico mais útil do exercício para quem pretende escolher um provedor em produção.

A capacidade de automatizar a coleta de dados, aplicar engenharia de *prompt* sofisticada e calibrar os resultados via modelos estatísticos em um único ambiente de programação oferece aos economistas e analistas de mercado um instrumento promissor para o monitoramento da conjuntura. Ainda há etapas naturais a percorrer antes de se tratar o índice como variável quantitativa estabelecida em modelos macroeconômicos — entre as quais se destacam a especificação preditiva (ata t contra Selic $t + 1$), o controle por estado macro contemporâneo e os testes de robustez a perturbações de *prompt*, todas listadas na seção anterior. Com essas extensões, o Índice de Tom do Copom tem potencial para se consolidar como um sinal antecedente útil na formulação de cenários e estratégias de investimento.

9 Referências

- Apel, Mikael, e Marianna Blix Grimaldi. 2012. *The Information Content of Central Bank Minutes*. Working Paper Series N. 261. Sveriges Riksbank.
- Bholat, David, Stephen Hansen, Pedro Santos, e Cheryl Schonhardt-Bailey. 2015. *Text Mining for Central Banks*. Handbook N. 33. Centre for Central Banking Studies, Bank of England.
- Blinder, Alan S., Michael Ehrmann, Marcel Fratzscher, Jakob De Haan, e David-Jan Jansen. 2008. “Central Bank Communication and Monetary Policy: A Survey of Theory and Evidence”. *Journal of Economic Literature* 46 (4): 910–45. <https://doi.org/10.1257/jel.46.4.910>.
- Caruso, M. A. 2026. *Brazil Macro Special Report: How Copom Tone Anticipates the Selic*. Special Report. Santander Brasil.
- Hansen, Stephen, e Sophia Kazinnik. 2023. *Can ChatGPT Decipher FedSpeak?* SSRN Working Paper N. 4399406. Federal Reserve Bank of Richmond. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4399406.

- Hansen, Stephen, e Michael McMahon. 2016. “Shocking language: understanding the macroeconomic effects of central bank communication”. *Journal of International Economics* 99 (Supplement 1): S114–33. <https://doi.org/10.1016/j.jinteco.2015.12.008>.
- Hansen, Stephen, Michael McMahon, e Andrea Prat. 2018. “Transparency and Deliberation Within the FOMC: A Computational Linguistics Approach”. *Quarterly Journal of Economics* 133 (2): 801–70. <https://doi.org/10.1093/qje/qjx045>.
- Hubert, Paul, e Fabien Labondance. 2021. “The Signaling Effects of Central Bank Tone”. *European Economic Review* 133: 103684. <https://doi.org/10.1016/j.eurocorev.2021.103684>.
- Loughran, Tim, e Bill McDonald. 2011. “When Is a Liability Not a Liability? Textual Analysis, Dictionaries, and 10-Ks”. *Journal of Finance* 66 (1): 35–65. <https://doi.org/10.1111/j.1540-6261.2010.01625.x>.
- Picault, Matthieu, e Thomas Renault. 2017. “Words are not all created equal: A new measure of ECB communication”. *Journal of International Money and Finance* 79: 136–56. <https://doi.org/10.1016/j.jimonfin.2017.09.005>.
- Woodford, Michael. 2003. *Interest and Prices: Foundations of a Theory of Monetary Policy*. Princeton University Press.